JAVADOC DOCUMENTANDO EL CÓDIGO

Cómo convertir comentarios en documentación útil

MATERIAL DE CÁTEDRA · USO ACADÉMICO

DSOO · 2026

¿QUÉ ES JAVADOC?

Documentación generada a partir del código

Javadoc es la herramienta oficial del JDK que genera documentación en HTML a partir de comentarios especiales escritos directamente en el código fuente.

Esos comentarios usan la sintaxis /** … */ y se ubican justo arriba de una clase, interfaz, método o atributo. Javadoc los lee y produce páginas navegables, iguales a las de la documentación oficial de Java.

El resultado no es un comentario más: es un contrato legible sobre qué hace cada parte del código, sin necesidad de abrir su implementación.

JAVADOC vs COMENTARIOS COMUNES

COMENTARIOS COMUNES

- **Explican al que lee el código**

Aclaran decisiones internas, un algoritmo complejo o dejan un TODO. Solo se ven en el código fuente.

- **No siguen un formato fijo**

Cada programador escribe lo que quiere, como quiere. No hay estructura ni herramienta que los procese.

- **No generan ningún artefacto**

Quedan atrapados dentro del .java. Nadie fuera del archivo se entera de que existen.

COMENTARIOS JAVADOC

- **Documentan la API pública**

Describen qué hace una clase o método para quien lo usa, sin necesidad de leer la implementación.

- **Siguen una sintaxis con etiquetas**

Usan /** */ y tags como @param, @return y @throws que una herramienta puede interpretar.

- **Generan documentación HTML navegable**

La herramienta javadoc los transforma en un sitio web con hipervínculos, igual al de la API de Java.

OTROS LENGUAJES (I)

JAVASCRIPT / TS — JSDoc

Comentarios /** */ sobre funciones con etiquetas @param y @returns. TypeDoc genera sitios HTML a partir de las anotaciones de tipos.

PYTHON — Docstrings + Sphinx

El lenguaje define docstrings entre triples comillas debajo de la definición. Sphinx o pydoc los convierten en documentación HTML.

C / C++ — Doxygen

Lee comentarios /** */ o //! en el código y genera documentación HTML o PDF, con diagramas de clases y de llamadas incluidos.

C# — XML Doc Comments

Comentarios /// con etiquetas XML (<summary>, <param>) que Visual Studio y herramientas como DocFX transforman en sitios navegables.

OTROS LENGUAJES (II)

RUST — rustdoc

Comentarios /// (o //! para el módulo) escritos en Markdown. cargo doc genera el sitio y compila los ejemplos de código como tests.

GO — go doc

No hay etiquetas especiales: el comentario justo antes de una declaración es su documentación. go doc la muestra en terminal o en un sitio web.

KOTLIN — KDoc + Dokka

Comentarios /** */ muy similares a Javadoc, con etiquetas @param y @return. Dokka genera el sitio HTML, incluso en proyectos mixtos Java/Kotlin.

PHP — phpDocumentor

Usa bloques /** */ con etiquetas al estilo Javadoc (@param, @return) para generar documentación de clases y funciones.

ESTRUCTURA DEL JAVADOC

Descripción + etiquetas, en ese orden

Un comentario Javadoc siempre empieza con /** y termina con */. La primera oración es el resumen: aparece en las tablas de índice, así que debe ser clara y terminar en un punto.

Después de la descripción vienen las etiquetas (tags), cada una en su propia línea, en el orden convencional: @author, @param, @return, @throws, @see, @since.

Dentro del texto se puede usar HTML simple (por ejemplo <code>) y etiquetas en línea como {@code} o {@link} para referenciar otra clase o método.

COMPONENTES BÁSICOS (I)



@param

Documenta cada parámetro del método, en el mismo orden en que aparece en la firma. Obligatorio para todo parámetro.

@return

Describe el valor devuelto por el método. Se omite en métodos void; es obligatorio en cualquier otro caso.

@throws / @exception

Documenta las excepciones que el método puede lanzar y en qué condición ocurre cada una.

@see

Crea una referencia cruzada hacia otra clase, método o campo relacionado, para saltar directamente ahí.

COMPONENTES BÁSICOS (II)

@author / @since / @version

Metadatos de la clase: quién la escribió, desde qué versión existe y en qué versión está el código actual.

@deprecated

Marca una API obsoleta y explica qué usar en su lugar. Suele combinarse con {@link} para señalar el reemplazo.

{@code}

Da formato de código a un fragmento dentro del texto, sin necesidad de escribir la etiqueta HTML <code>.

{@link}

Inserta un hipervínculo hacia otra clase o método dentro del texto de la descripción, para navegar la documentación generada.

JAVADOC EN LA BIBLIOTECA



```java
/**
 * Indica si hay copias
 * disponibles para préstamo.
 * @return true si hay una
 *   copia libre
 */
public boolean
    tieneEjemplaresDisponibles()
```

```java
/**
 * Registra el préstamo de un
 * libro a este socio.
 * @param libro el ejemplar
 *   que se entrega
 */
public void
    registrarPrestamo(Libro libro)
```

LIBRERÍAS: Commons Lang3



```java
/**
 * Tests if a CharSequence is
 * empty, null, or blank.
 * @param cs the value to check
 * @return true if blank
 */
public static boolean
    isBlank(CharSequence cs)
```

```java
/**
 * Capitalizes a String, the
 * first char to title case.
 * @param str value to capitalize
 * @return the capitalized value
 */
public static String
    capitalize(String str)
```

LIBRERÍAS: Guava y JUnit



```java
/**
 * Returns consecutive sublists
 * of a list, same size each.
 * @param list list to split
 * @param size size per sublist
 */
public static <T>
  List<List<T>> partition(...)
```

```java
/**
 * Asserts execution of the
 * executable throws expectedType.
 * @param expectedType exception
 * @param executable code to run
 */
static <T extends Throwable>
  T assertThrows(...)
```

CÓMO GENERAR LA DOCUMENTACIÓN

De los comentarios al sitio HTML

El comando javadoc, incluido en el JDK, recorre los archivos .java y genera un sitio HTML navegable con todas las clases, métodos y sus comentarios.

Los IDEs lo integran directamente: en Eclipse y NetBeans es Project → Generate Javadoc; en IntelliJ, Tools → Generate JavaDoc. Maven usa el maven-javadoc-plugin para generarlo como parte del build.

El resultado es el mismo tipo de sitio que la documentación oficial de la API de Java: un índice navegable, una página por clase, con métodos, parámetros y enlaces cruzados.

MENSAJE FINAL

"La documentación es el puente entre quien escribe el código y quien lo usa."

Javadoc convierte los comentarios en documentación real: generada, consistente y siempre junto al código.